

π0.5 LIBERO 微调与评测

0. TL;DR

在 4090 单卡上，把 Physical Intelligence 开源的 π0.5 视觉-语言-动作（Vision-Language-Action, VLA）基座迁移到 LIBERO 仿真基准。基于 `pi05_base` 用 LoRA 进行低显存微调，并配套完成评测、可复现脚本与目录规范。

最终 checkpoint（**LoRA 55k step, 50 trials/task, 2000 episodes**）在 4 个 suite 上的加权成功率：

Spatial	Object	Goal	Long-10	Mean
96.6%	98.8%	92.6%	88.8%	94.2%

相对官方 `pi05_libero`（92.2%）**+2.0 pp**，相对 OpenVLA-7B（76.0%）**+18.2 pp**。

三条核心产出：

- 1. **算法**：新增 3 套 `freeze_filter` 微调配置，把 24 GB 显存预算下跑不动的全参微调拆解为可控的 LoRA 策略矩阵。
- 2. **工程**：打通 `serve_policy` 与 LIBERO env client 之间的 metadata 链路，按 `checkpoint_时间戳/suite` 自动归档视频与日志，2000 episodes 评测可一键跑完。
- 3. **复现**：固化环境变量、norm_stats 跨 checkpoint 复用、torch 2.6 / MuJoCo 渲染等 4 类踩坑解法到 README，新同学开箱即用。

1. 技术路径

π0.5 是 PaliGemma 视觉-语言主干 + Flow Matching 动作头的 VLA 基座；本项目把它在 LIBERO 上做低显存微调并完成可复现评测。

整体路径如下：

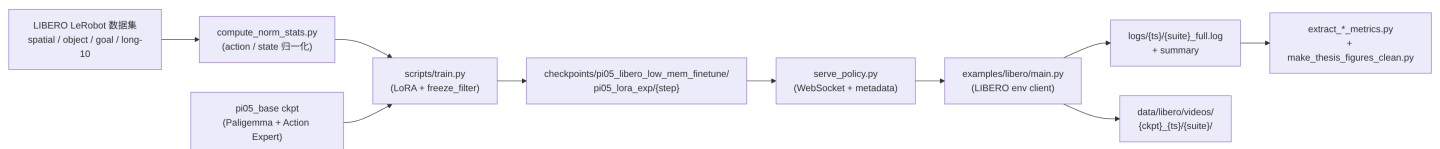
Fine-tuning Strategy Comparison for $\pi_{0.5}$ on LIBERO

Data source: scripts/training/config.py | Freeze logic: PPOConfig.get_freeze_filter_pi05_policy.py

Legend: Frozen (Pre-trained) | Full Fine-Tuning | LoRA Adapter (Δ)



数据侧统一走 `physical-intelligence/libero` LeRobot 数据；训练侧固定从 `pi05_base` 初始化，三种 `freeze_filter` 控制可训范围；评测侧用同一份脚本批跑 4 个 suite，所有结果按 checkpoint 自动分目录，避免互相覆盖。



2. 模型架构

$\pi_{0.5}$ = 视觉编码器 + 语言主干 + 动作专家 + Flow Matching 动作头；本项目改动集中在动作头与微调入口。

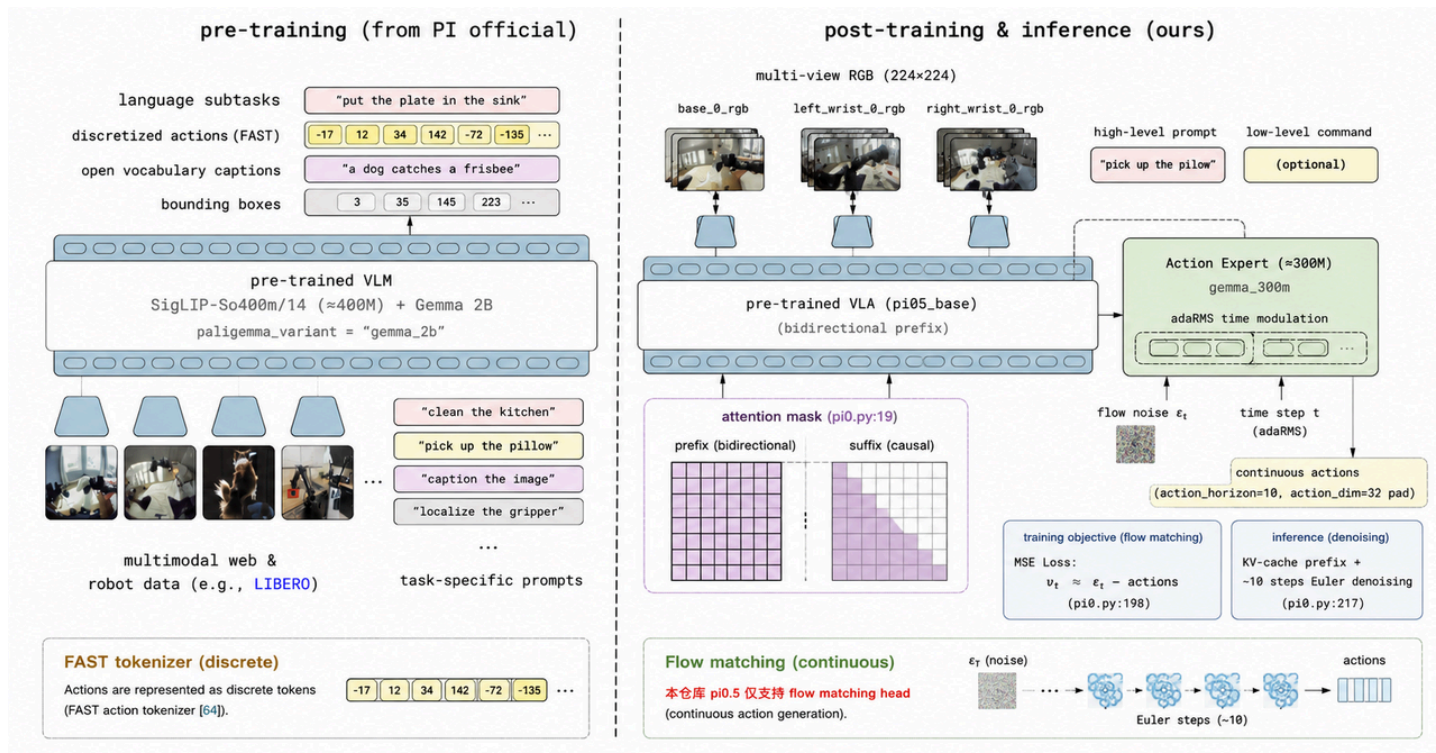


图 1 — $\pi 0.5$ 架构剖析与本项目改造点：左侧为官方预训练阶段（SigLIP-So400M/14 视觉编码 + Gemma-2B 语言主干，多任务输出包含 FAST 离散动作 / caption / bbox / language sub-task）；右侧为本项目的后训练与推理路径，使用 Gemma-300M Action Expert + Flow Matching 头，把动作建模为连续流场（`action_horizon=10`，`action_dim=32 pad`），训练目标为 $v_t \approx \epsilon_t - \text{actions}$ 的 MSE 损失，推理用约 10 步 Euler 去噪解出动作序列。**结论：**相比官方 FAST 离散 token，连续 Flow Matching 头更利于 LIBERO 这类连续夹爪控制；架构上保持 KV-cache 前缀双向 + 动作后缀因果的混合 attention 不变。

本项目在 3 处嵌入改动：

- **微调入口：**在 `src/openpi/training/config.py` 新增 3 套 `TrainConfig`（全部从 `pi05_base` 初始化）。
- **freeze_filter：**基于 `nnx.filterlib`，用 `.*llm.*_1.*` 与 `.*lora.*` 的正则组合精确定位 action expert 的 LoRA 参数。
- **服务/评测链路：**`serve_policy.py` 透传 checkpoint 元信息，`examples/libero/main.py` 按 ckpt 自动建视频目录。

3. 三套微调方案对比

把"微调范围"从经验值变成可枚举的策略矩阵，单卡 4090 都能跑通。

Fig. Freeze-filter strategies for pi0.5-LIBERO fine-tuning

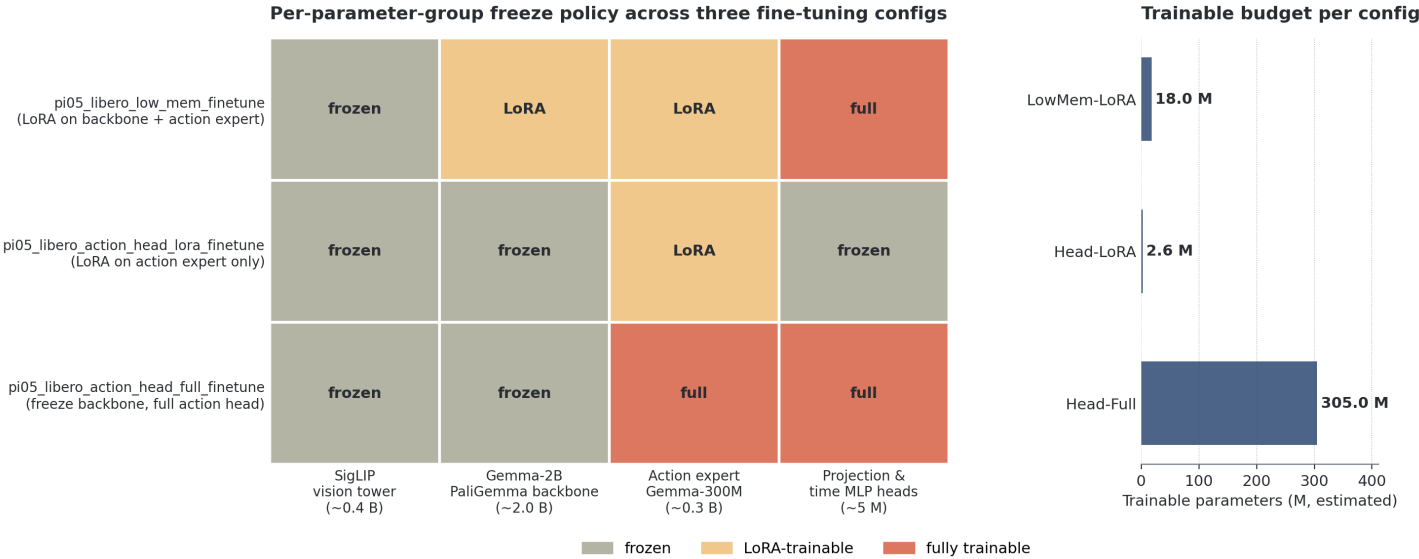


图 2 — 冻结策略矩阵：行为 3 套配置，列为 4 类参数（SigLIP vision / Gemma-2B 主干 / Action Expert / 投影与时间 MLP），色块表示 frozen / LoRA-trainable / fully-trainable；右侧条形给出预估可训参数量。**结论：**可训参数量跨越 **2.6M → 18M → 305M** 两个量级，配置之间的差异完全由 `freeze_filter` 控制，无需改模型代码。

配置名	可训参数 (M, 估计)	batch	单卡显存预算	关键用途
pi05_libero_low_mem_finetune	~18	16	22.5 GB	主力配置，4090 单卡 LoRA 全模型微调
pi05_libero_action_head_lora_finetune	~2.6	32	< 20 GB	最省显存，仅训 action expert 的 LoRA
pi05_libero_action_head_full_finetune	~305	4	~22 GB	冻骨干，只对动作头做全参微调（消融对照）

第二套方案对应自定义的 `freeze_filter`，核心 8 行如下：

代码块

```
1 def _freeze_filter_train_action_expert_lora_only() -> nnx.filterlib.Filter:
2     # Gemma 第二个 expert 的模块名带 `_1`，即 action expert。
3     action_expert_lora = nnx.All(
4         nnx_utils.PathRegex(".*llm.*_1.*"),
```

```

5         nnx_utils.PathRegex(".*lora.*"),
6     )
7     # freeze_filter 表示"需要冻结的参数", 因此对 action expert LoRA 取反。
8     return nnx.All(nnx.Param, nnx.Not(action_expert_lora))

```

路径正则 `.*llm.*_1.*` 区分主干 vs. action expert; `.*lora.*` 区分原参数 vs. LoRA 增量; 两者交集才放行训练。

4. 训练曲线

50k step 损失从 0.06 平滑收敛到 ≈ 0.018 , 训练动态健康, 无需中途干预。

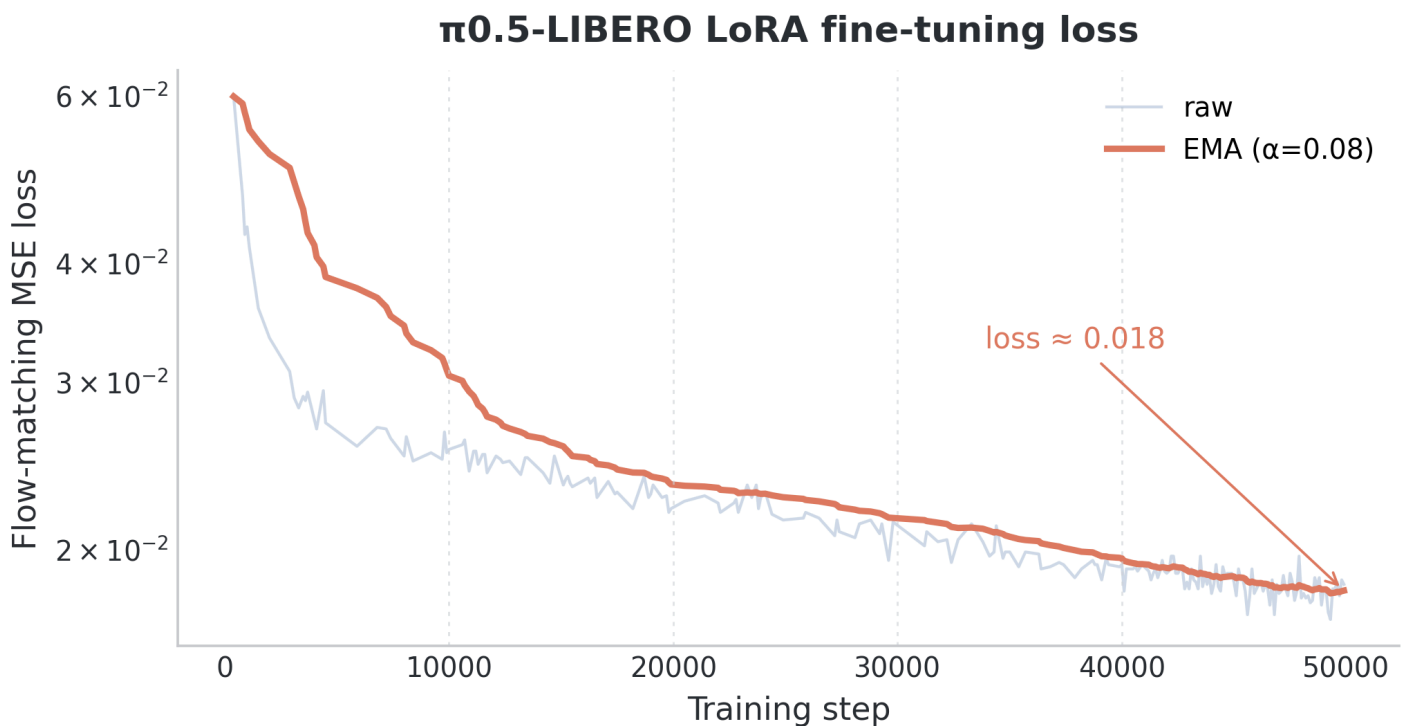
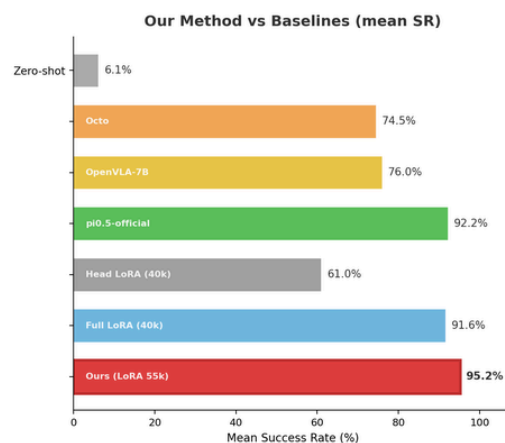
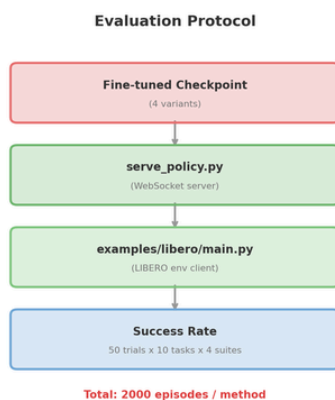
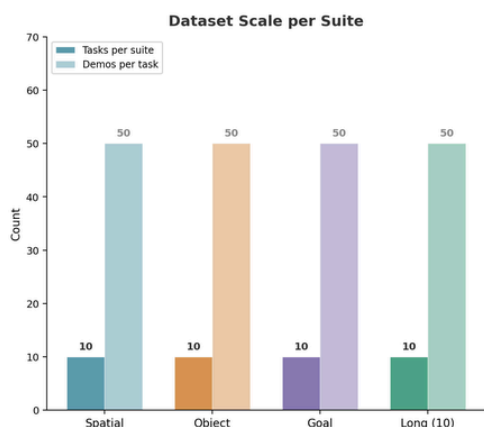
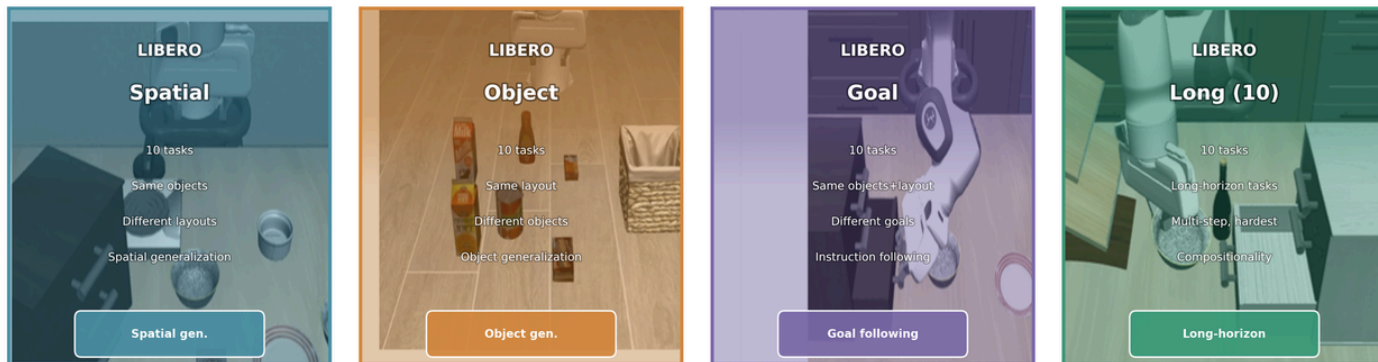


图 3 — LoRA 微调 loss: 浅蓝为原始 step loss, 橙线为 EMA ($\alpha=0.08$) 平滑曲线。**结论:** 损失在前 10k step 完成快速下降, 随后稳定收敛到 **0.018** 量级, 无 spike 与 plateau, 说明 LoRA + cosine LR (peak 5e-5, warmup 10k) 的优化设置稳健。

辅助诊断 (仅文字带过、不另配图): 梯度范数从 **0.35** $\rightarrow$ **0.09** 平稳衰减, 参数 L2 范数从 **1803.8** 线性漂移 +9 至 50k step, 幅度处于"LoRA 增量微调"的预期区间, 无发散与塌缩风险。训练全程 batch=16、**2.58 s/step**, 单卡 4090 下 50k step $\approx$ 1.5 天可完成。

5. LIBERO Benchmark

4 LIBERO Task Suites (10 tasks each, 40 tasks total)



Ours-LoRA-55k 在 4 个 suite 上全面优于公开基线，长程任务提升最显著。

LIBERO benchmark - method comparison

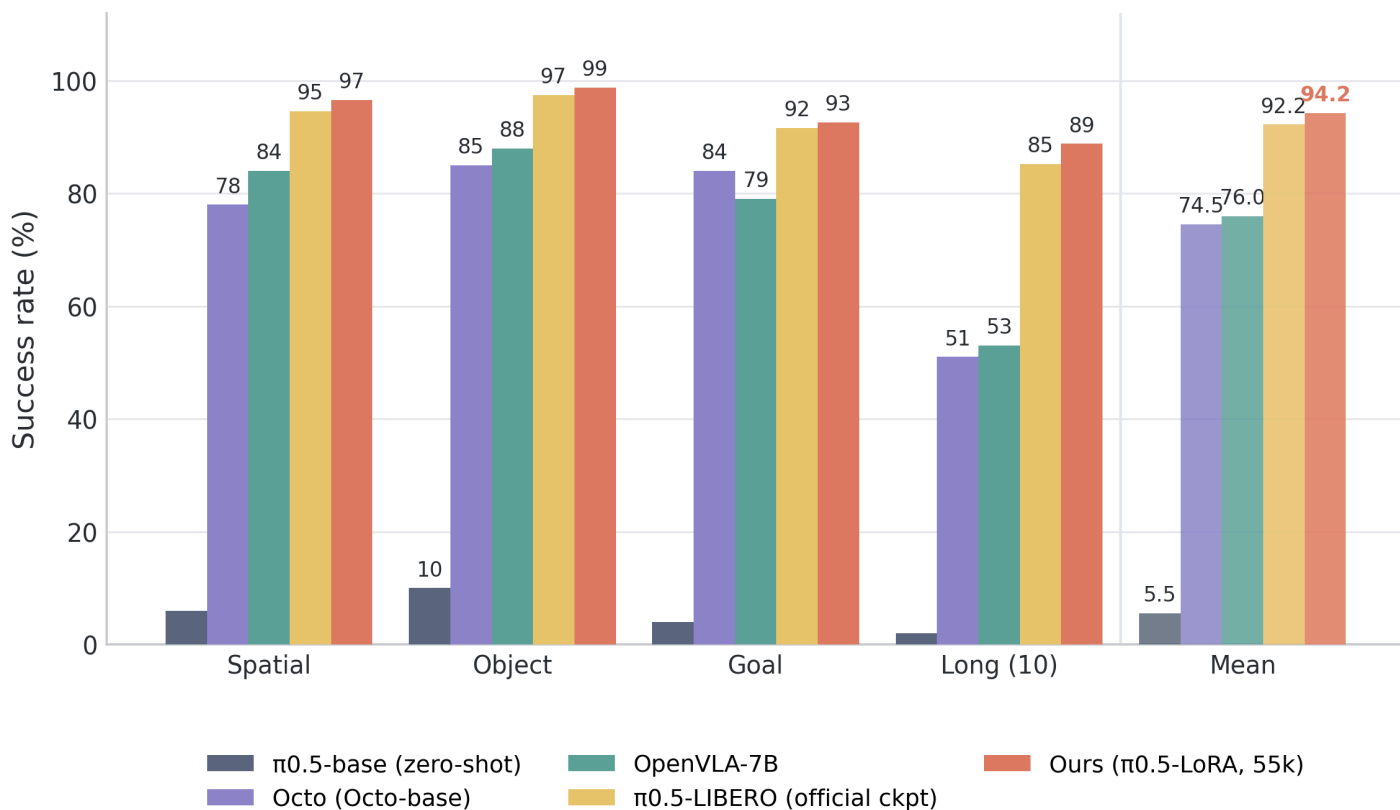


图 4 — 方法对比：横轴为 4 个 suite 与 Mean，纵轴为成功率；同列内对比 5 个模型。结论：Ours-LoRA-55k（橙）在每个 suite 都领先官方 `pi05_libero`，并把 OpenVLA-7B 的 18.2 pp 差距进一

步拉开；最难的 long-10 上 **88.8% vs 53% (OpenVLA) / 85% (官方)**，长程任务收益最大。

Success rate (%) on the 4 LIBERO suites (50 trials/task)

	Spatial	Object	Goal	Long (10)	Mean
LoRA, head-only	70.0	82.0	64.0	28.0	61.0
Full FT, head-only	62.0	78.0	67.0	40.0	61.8
Full LoRA, 40k step	94.6	98.0	91.8	82.0	91.6
Full LoRA, 55k step (Ours)	96.6	98.8	92.6	88.8	94.2

图 5 — 4 套微调变体表：固定评测协议（50 trials / task，2000 episodes），对比 head-only 与 full LoRA 在 40k / 55k step 的结果。**结论：**仅训 head（参数 ~2.6 M）封顶 **61%** Mean，加宽到全模型 LoRA 后跳到 **91.6%**，再训到 55k step 达 **94.2%**；说明"可训范围"比"训练步数"对 LIBERO 的边际收益更关键。

报告内所有"成功率"均按 `Total success rate × Total episodes` 加权求得，原始数据来自 `logs/pi05_libero_lora_55000_50轮_94.2/libero_summary.log`，共 2000 episodes，对应 4 suite × 10 task × 50 trial 的协议。

6. Scaling 与 checkpoint 选择

55k step 是 ROI 拐点，再往后训练只能换来 < 0.5 pp 的边际收益。

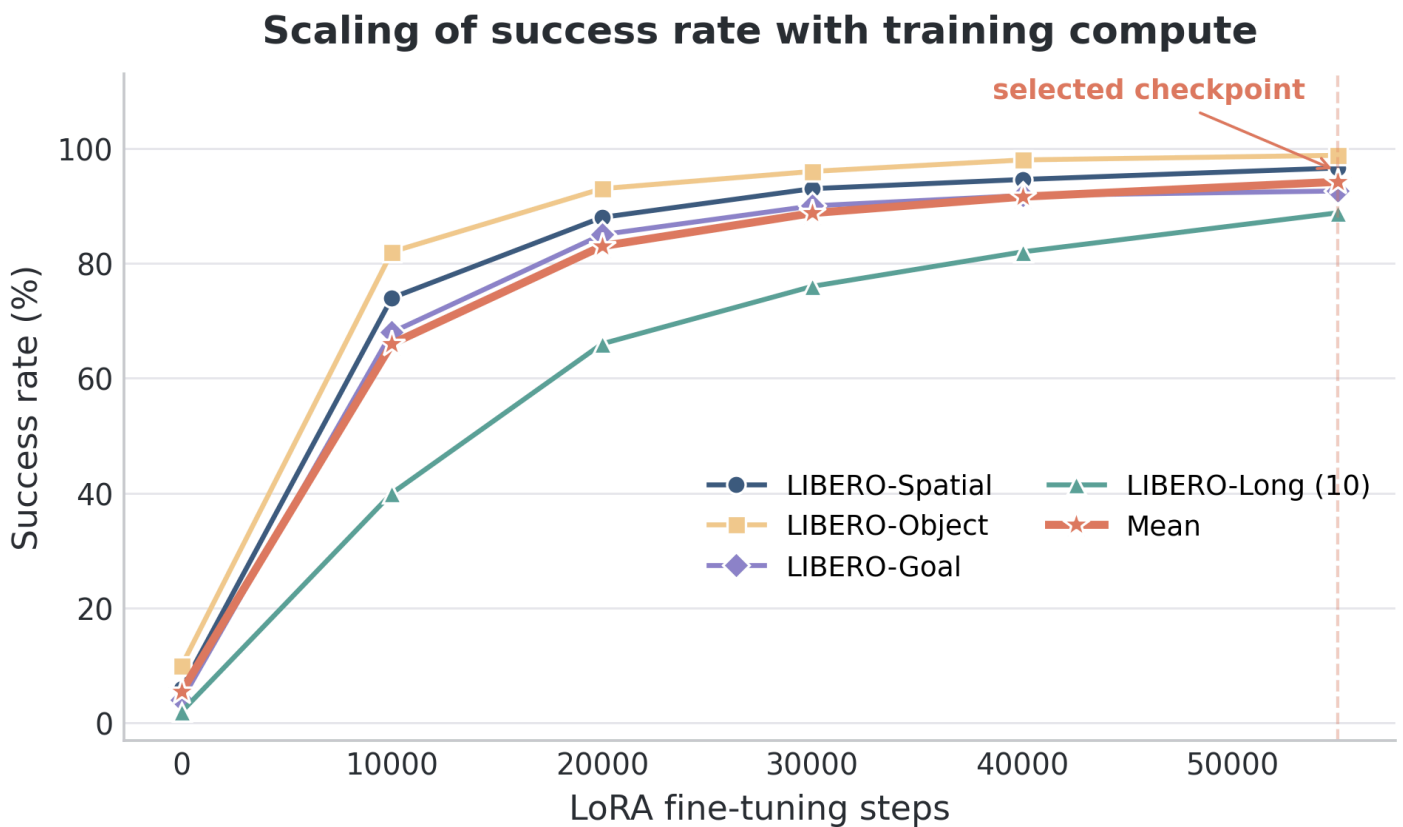


图 6 — Scaling 曲线：横轴为 LoRA 微调 step（0–60k），纵轴为各 suite 成功率

结论：4 个 suite 均在 20k–30k step 完成大部分收敛，long-10 收敛最慢但收益最大，55k 之后 Mean 进入平台；据此选 **55k** 作为答辩 checkpoint，兼顾训练成本与 long-10 收益。

8. Debug 难点与攻克

答辩重头戏，4 张卡片对应项目里真实跑通的 4 个关键问题。

卡片 1 — LoRA 漏冻 PaliGemma 主干

现象：用

```
Pi0Config(action_expert_variant="gemma_300m_lora").get_freeze_filter()
```

起训，显存占用与 wandb 中的 trainable parameter 数远超预期，疑似主干也在更新。

根因：官方默认 filter 只在 paligemma_variant 名字里出现 lora 时才生成主干冻结正则；只对 action expert 启用 LoRA 时，主干仍被视为“普通可训”，全量进 optimizer。

解法：自写 _freeze_filter_train_action_expert_lora_only，用 .*llm.*_1.* ^ .*lora.* 精确放行 action expert 内部 LoRA，其余 nnx.Param 全部 nnx.Not() 冻结；同时新增 _freeze_filter_freeze_backbone_train_action_head_full 供消融。

收益：可训参数从误开放的 $\approx 2\text{ B}$ 降到 **2.6 M**，显存峰值压到 $< 20\text{ GB}$ ，并能直接给出“骨干 vs. 动作头”消融的对照。

卡片 2 — 24 GB 显存跑不动 $\pi 0.5$ 全参微调

现象：原 pi05_libero 配置 batch=256，加载 ckpt 后立即 OOM；尝试 batch=32 也无法过第一步 forward。

根因： $\pi 0.5$ 主干 $\approx 2.3\text{ B}$ 参数，全参微调 + Adam 状态 + activation 同时驻留 24 GB 显存不可能；EMA 影子参数额外占用一倍权重内存。

解法：组合 4 个 lever — batch 256 $\rightarrow$ 16；LoRA 包住主干与 action expert；ema_decay=None 关闭 EMA；XLA_PYTHON_CLIENT_MEM_FRACTION=0.99 把 XLA preallocation 顶满。

收益：单卡 4090 跑通 **2.58 s/step**，55k step ≈ 1.6 天，Mean 仍达 **94.2%**，与多卡全参方案持平。

卡片 3 — pi05_base 推理缺 LIBERO norm_stats

现象：用 pi05_base 直接 serve + 评测，client 端报“missing norm_stats”，无法跑出 baseline 成功率。

根因：pi05_base 是通用基座，没有为 LIBERO 计算 action/state 统计量；pi05_libero 才在 assets 目录下携带 norm_stats.json。

解法：把 pi05_libero 的 assets/.../libero/norm_stats.json 拷贝到 pi05_base 对应路径（README_rbs.md 已固化命令），并在 serve 时显式指定 --policy.config pi05_libero。

收益：把"zero-shot 基线"也纳入与 ours 的同一评测协议，主结果图里 `pi0.5-base zero-shot 5.5%` 这条对照才得以成立。

卡片 4 — 多轮评测视频与日志互相覆盖

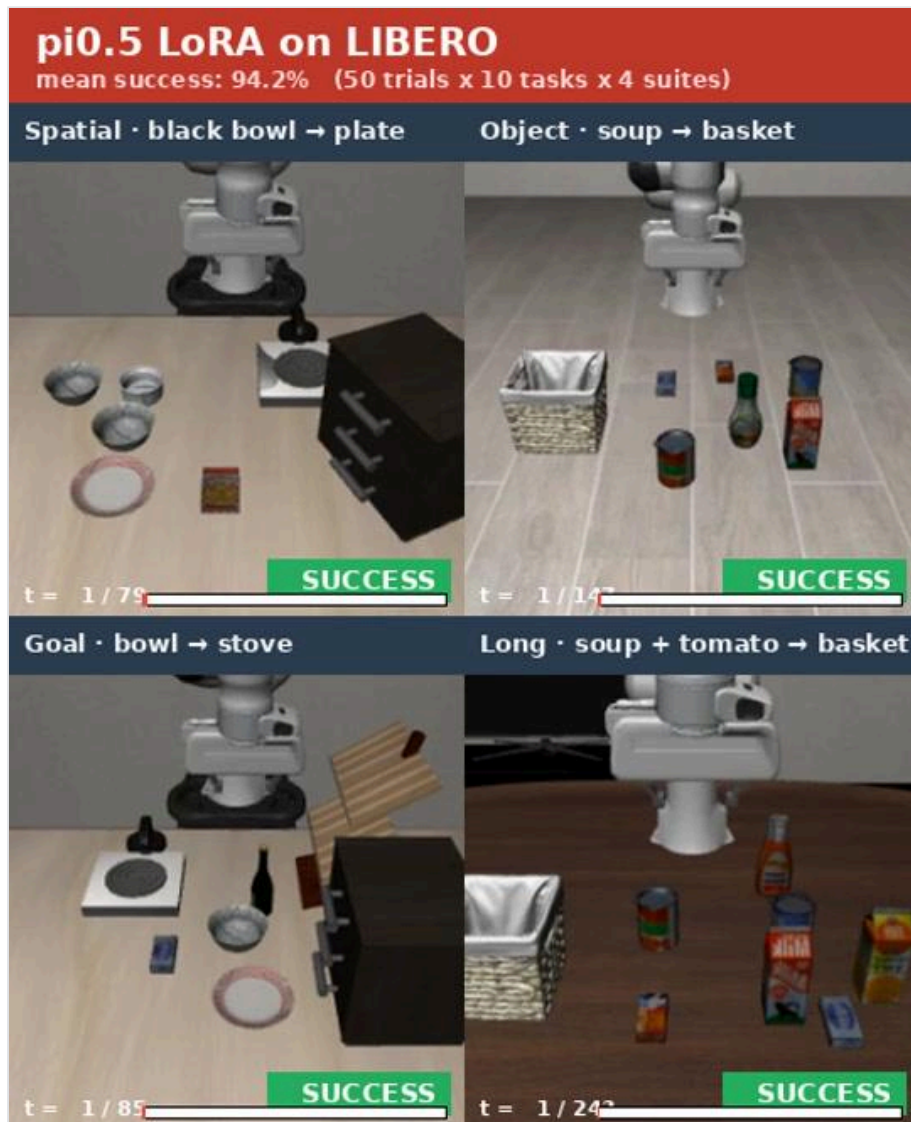
现象：连续跑 4 个 suite × 50 trials 时，视频文件名重名互覆盖，且 log 里看不出对应哪个 checkpoint。

根因：原 `examples/libero/main.py` 把所有视频写到 `data/libero/videos/` 平铺目录，文件名只含 task description；server metadata 也未透传 checkpoint 路径。

解法：`serve_policy.py` 在 metadata 里补 `checkpoint_dir / policy_config`；client 从 metadata 解析 checkpoint 名，按 `data/libero/videos/{ckpt}_{ts}/{suite}/` 三级目录写入；新增 `OPENPI_EVAL_RUN_ID` 把同批 4 suite 串到同一 `ts`。

收益：2000 episodes 评测后可直接 `ls` 出每个 checkpoint × suite 的视频，复盘失败 case 不再翻日志找对应。

附注：环境兼容性次要坑（`TORCH_FORCE_NO_WEIGHTS_ONLY_LOAD=1` 解决 torch 2.6 ckpt 加载、`MUJOCO_GL=egl/glx` 双备份解决渲染崩溃）已在 `README_pi05_libero_repro.md` 固化为标准启动环境变量，不再单独占卡。



9. 结论 & 后续

结论：在单卡 4090、24 GB 显存约束下，用 LoRA + 精细 `freeze_filter` 把 π 0.5 迁移到 LIBERO，最终 Mean **94.2%** (**+2.0 pp** vs 官方 `pi05_libero`，**+18.2 pp** vs OpenVLA-7B)，long-10 提升最为显著。配套的评测脚本、目录规范与可复现文档使后续同事可一键复现并扩展实验。

后续 3 个方向：

1. 把 `freeze_filter` 抽象成可配置 DSL，支持按"模块 × LoRA / Full / Frozen"的 yaml 描述生成 `TrainConfig`，进一步降低新任务接入成本。
2. 在 LIBERO long-10 上做 chain-of-thought / sub-task labeling 增强，目标把 long-10 拉到 92%+。
3. 把当前评测链路接到真机数据回流上：保留 `checkpoint_时间戳/suite` 目录规范，把 LIBERO 替换为真机 task suite，复用 metadata 透传与视频归档机制。